CodeStorm24

OffGridGuru

Hackathon Summary Report: an offline engine that turns any concept into a narrated visual lesson.

Type a concept. In two to three minutes, on an ordinary CPU-only laptop with no internet and no cloud AI, get a 30 to 90 second lesson with animated diagrams, highlighted code and captions in English, Hindi or Tamil. It plays in the browser and is saved as a 720p MP4.

How does it work? यह कैसे काम करता है? இது எப்படி வேலை செய்யுது?

Team

Team Rogue

Members

Bhuvaneshwer S

Jeyanth S

Repository

github.com/Jeyxnth/OffGridGuru (branch main)

Report date

28 October 2026 · Built with Llama (Meta Llama 3.2)

1 Overview

Most explainer videos are English, need an internet connection, and take a person hours to make. OffGridGuru makes a short, narrated, visual lesson on any topic, in the learner's own language and script, entirely on the learner's own laptop.

0
cloud services or API keys at runtime: fully offline

3
languages, each in its own script: English, Hindi, Tamil

2–3 min
per 60 s lesson, MP4 included, on a CPU-only laptop

331
automated backend tests, plus 24 frontend tests

The problem

- Learners in India often study in Hindi, Tamil or another Indian language, but most visual explainers are in English.
- Cloud AI tools need a fast connection, an account and money, and they send the learner's data away.
- Making a narrated, animated lesson by hand takes hours per topic.

Our answer

- A small local LLM plans the lesson as structured JSON; our own components draw it.
- Local models translate it (IndicTrans2) and voice it (Kokoro, AI4Bharat Indic-TTS).
- The audio decides the timing, so picture, captions and voice never drift.
- Every stage has a fallback, and the screen says when one was used, so the lesson always plays.

How a learner uses it

1. **Ask.** Type a concept (for example "How does binary search work?"), then pick the level, language, voice, speed and length (30–90 s).
2. **Watch it being made.** The stages tick off: planning, translating, voicing, timing, exporting. Each shows its real time.
3. **Learn.** The player opens as soon as the narration is ready: scenes animate in step with the voice, and captions are in the lesson's script.
4. **Check.** A short quiz at the end checks what the lesson taught, with instant feedback.
5. **Keep and share.** Download the MP4 and the captions file, print a one-page summary, re-voice the same lesson in another language, or try a suggested next topic.

2 Architecture

A FastAPI backend runs five stages, one at a time, and unloads each model after its stage, so the app fits an 8 GB laptop. A React + TypeScript frontend draws the lesson as SVG and CSS. The `shared/` folder (JSON schemas, sample lessons, glossary and bundled audio) is the contract between the two halves.

1
Plan
Llama 3.2 3B through Ollama on 127.0.0.1 writes the lesson as JSON: 3–6 scenes (title card, diagram, code walkthrough), a quiz and two next topics. It is checked against the schema and content rules, with 2 attempts.

Fallback: the closest of four built-in lessons

2
Translate
IndicTrans2 (en→hi, en→ta) on the CPU. A glossary keeps code, numbers and technical terms in English. The model is unloaded afterwards.

Fallback: a pre-translated sample, or English narration

3
Voice
Kokoro (English, Hindi) and Indic-TTS (Tamil), one sentence at a time: normalised, trimmed and cached.

Fallback: the sample's bundled audio

4
Time
Each scene lasts as long as its sentences plus short pauses. This produces `timeline.json`, `lesson.wav` and `captions.srt`.

Audio-first: only this stage sets durations

5
Render
The player draws the scenes as a function of the timeline and the time. For the MP4, headless Chromium steps through the same page and FFmpeg encodes the frames.

Fallback: keyframe export; the lesson always plays

Every stage records its wall time and peak memory in `timings.json`; the player shows them in a speed panel.

Technology stack

Layer	What we used
Backend	Python, FastAPI, Pydantic v2, jsonschema; one worker thread with a job queue; Server-Sent Events for live status
Planner	Meta Llama 3.2 3B Instruct (Q4_K_M) served by Ollama; output forced to a JSON format; low temperature
Translation	AI4Bharat IndicTrans2 en-indic dist-200M (PyTorch, CPU)
Speech	Kokoro-82M (ONNX Runtime) for English and Hindi; AI4Bharat Indic-TTS (FastPitch + HiFi-GAN) for Tamil; Indic Parler-TTS as an opt-in Tamil voice
Frontend	React 18, TypeScript (strict), Vite; SVG and CSS scenes only, with no animation or 3D libraries, so it runs on a modest phone
Video	Playwright + Chromium for capture; FFmpeg + libx264 for an H.264/AAC MP4 with a soft caption track; ffprobe checks every file

The rules we built to

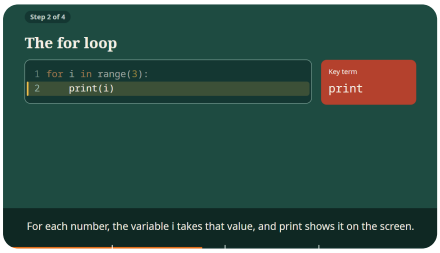
- **No cloud AI, fully offline.** No external API, CDN, web font or telemetry. Only the one-time model download script uses the network; a check script proves that nothing else does.
- **CPU-only, 8–16 GB RAM.** One model in memory at a time, unloaded after its stage.
- **Data, not code.** The model only writes JSON. Nothing it writes is run or rendered as HTML, and code is shown as text.
- **Audio-first timing.** Durations come from the real narration, so voice and picture cannot drift.
- **Deterministic rendering.** Every frame depends only on the timeline and the time, so the player and the MP4 match frame for frame.
- **The demo never breaks.** Every stage has a fallback, and the screen always says when one was used.

3 The lesson: player and video

The stage is a fixed 1280×720 chalkboard, scaled to fit any screen. The player and the MP4 export draw it with the same code. Three scene types cover most topics: a **title card**, a **diagram** (row, cycle or stack, whose items appear as the narration reaches them) and a **code walkthrough** (lines highlighted as they are explained, with a key-term note). Each script uses its own Noto font, with taller lines for Devanagari and Tamil, so vowel marks are never clipped.



The player: the chalkboard stage, keyboard-friendly controls (Space, ←, →, R), MP4 and caption downloads, and the speed panel.



English: a code walkthrough with the marigold highlight and the key-term note.



Hindi: the title card, with Devanagari serif heading and captions.



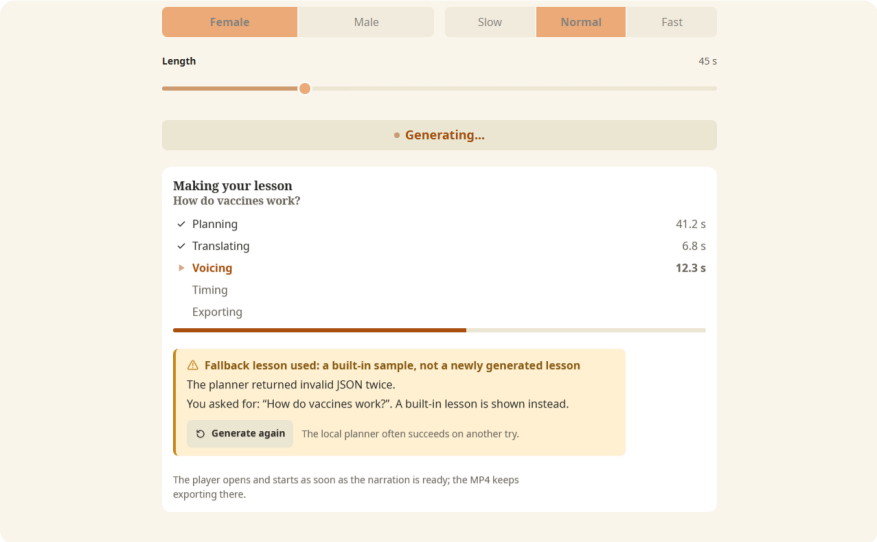
Tamil: a diagram; the newest item has a saffron edge.

The exported MP4

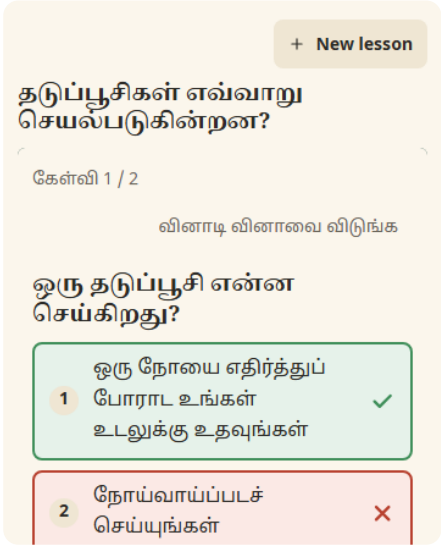
- 1280×720 H.264 (yuv420p) with AAC audio and a soft subtitle track tagged eng, hin or tam. Captions are also burned in.
- Smooth export at 15 frames per second, which sends only frames that change and repeats still ones. If it fails or runs over its time budget, a keyframe export runs instead.
- Every file is checked with ffprobe: size, codecs and duration within 0.2 s of the timeline. A cut-off file is never offered for download.

4 Features

Feature	What it does
Concept to lesson	Any topic, beginner or intermediate, 30–90 s, female or male voice, slow, normal or fast speed. Live stage progress with real timings and a queue position.
Three languages	English, Hindi and Tamil, each in its own script and font. Code, numbers and technical terms stay in English inside translated narration. The Tamil samples are written in spoken classroom Tamil.
Same lesson in another language	Re-voices the stored English plan into another language: translated, voiced, timed and exported again, without planning again.
Reuse cache	An identical request returns the finished lesson at once (measured at 27 ms). Fallback and short lessons are never cached.
Mini quiz (bonus)	Two or three multiple-choice questions on the lesson's own scenes, written in the same planner call. Instant feedback, an explanation and a score. Keys 1–4 and Enter. A bad question is dropped and never costs the lesson.
Learning habit (bonus)	Two "try next" topics, a daily streak and a revise list. A low quiz score adds the lesson to the list automatically. Everything stays in the learner's browser, with no account.
One-page summary (bonus)	An A4 summary built only from the lesson's JSON: key points, code, diagrams, the quiz as a worksheet with an answer key, and the next topics. It prints from the browser or downloads as a server-made PDF, and shrinks step by step until it fits one page.
Honest fallbacks	A clear notice whenever a sample, English narration or a shorter-than-asked lesson is shown, with a **Generate again** button. Generate again stops the old lesson's MP4 export, so the new one starts at once.
Speed panel	The seconds and peak memory of every stage, read from the lesson's own `timings.json`.
Accessible design	WCAG AA contrast on every text pair, 44 px touch targets, visible focus, full keyboard control, a 360 px phone layout, and reduced-motion support.



Making a lesson: live stages, and an honest fallback notice with Generate again (shown here with a simulated planner failure).



The quiz in Tamil at phone width (360 px).

5 Reliability: what happens when something fails

A live demo on borrowed hardware has to survive a stopped Ollama, a missing model or a slow export. Every stage has a planned answer, and each one is exercised by an automated failure drill.

If this fails	This plays instead	The screen says
Planner: Ollama stopped, model missing, timeout, or two invalid plans	The closest of four built-in lessons, in the chosen language	"Fallback lesson used", the reason, and Generate again
Planner: the last plan is short for the length asked, but at least 30 s long	That plan, as a shorter lesson on the topic asked	"Shorter than you asked: about N s instead of M s"
Translation: model missing or bad output	A sample's checked translation, or the lesson in English	"Translation unavailable"
Voice: model missing, out of memory, engine error	The closest sample, voiced from bundled audio	"Fallback lesson used: Voice generation failed"
Smooth MP4 export	The keyframe export (one frame per change)	The MP4 still downloads
Both exports	The lesson still plays in the browser	"MP4 export failed" and Retry export
Quiz: bad questions	Bad questions are dropped; with fewer than 2 left, there is no quiz	Nothing: the lesson ends normally

Safety of model output

The planner's reply is untrusted text. It is parsed as JSON, cleaned, and screened for markup and scripts. It is then validated against the shared schema and content rules: 3–6 scenes, a word budget for the length, short distinct diagram labels, code of at most 12 lines of 50 characters, and English only. A rejected first plan is retried once with the errors listed. Nothing from the model is ever executed or inserted as HTML; React renders all of it as plain text.

Offline proof

`scripts/check_offline.py` confirms that every model, voice, font, tool and the capture browser are on disk. It finds no external URLs in the source or the built frontend, and it shows that nothing reaches the internet: outbound sockets are blocked, only loopback is allowed, and no connection was attempted.

6 Results

Submission videos (made by the app, offline)

Planner llama3.2:3b, beginner, 60 s target. Each Hindi version re-voices the same English plan.

Topic	Language	File	Length	Export
How does binary search work? (our topic)	English	`how-does-binary-search-work_en.mp4`	41.7 s	smooth
	Hindi	`how-does-binary-search-work_hi.mp4`	57.6 s	smooth
Explain how caffeine works (judges' topic)	English	`explain-how-caffeine-works_en.mp4`	41.0 s	smooth
	Hindi	`explain-how-caffeine-works_hi.mp4`	61.0 s	smooth

Speed on the development laptop

Intel Core i7-13650HX (14 cores, 20 threads), 24 GB RAM, Windows 11. The planner was forced onto the CPU (`OLLAMA_NUM_GPU=0`). Each run is a fresh 60 s lesson. The budget was 5 minutes.

Language	Plan	Translate	Voice + timing	MP4 export	Total	Peak RAM
English (58.0 s lesson)	45.2 s	–	11.6 s	53.5 s	**112 s**	2.66 GB
Hindi (62.2 s lesson)	44.8 s	18.7 s	18.0 s	60.1 s	**144 s**	2.66 GB
Tamil (77.4 s lesson)	46.4 s	10.8 s	37.0 s	76.3 s	**173 s**	2.65 GB

Peak RAM is the Ollama planner during planning; the app itself peaks at about 2.1 GB during export. With a GPU, planning takes about 10–25 s.

Tests and drills

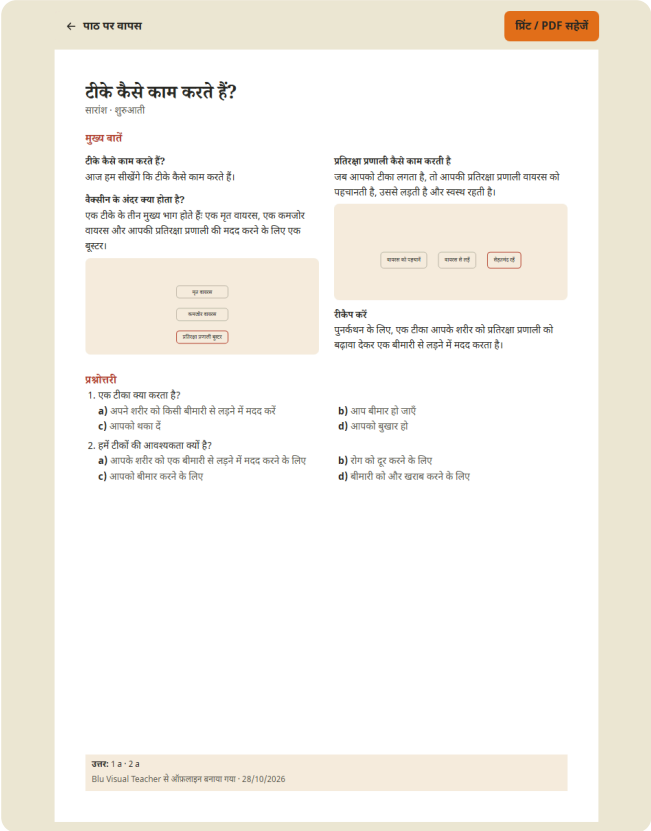
Check	Result
Backend test suite (pytest)	**331 tests.** On the final full run 330 passed; the one failure is a timing gate (sample render under 60 s) that passes when run on its own and runs over only after about 100 s of other heavy tests on the same laptop.
Frontend tests (Vitest), TypeScript strict typecheck	**24 passed; typecheck clean**
Failure drills (planner down, garbage plans, missing translation, missing voices, export failure)	**6 / 6 passed**; each showed the right notice and still played
Offline matrix: 3 topics × 3 languages × 2 levels, network blocked	**18 / 18 passed**; one case used the planner fallback, and its notice was shown as designed
Phone check (emulated, CPU slowed 4× and 6×)	**Smooth**: median 7 ms per frame; 0.3% and 1.3% of frames over 33 ms
Clean clone from the README	**Set up and passing in 16 minutes**; it found and fixed three setup problems

7 Design

The look is "Chalkboard and Paper". The app is cream notebook paper with charcoal ink and saffron actions. The lesson itself is a green chalkboard with chalk-white text, saffron for progress and the newest item, a marigold code highlight and a terracotta key-term note. Headings are set in Noto Serif. The saffron and green echo Indian colours without copying the flag. There are no gradients, glows or shadows: shapes are flat with soft rounded corners.

Three design passes

- **D1, audit:** spacing and type tokens, WCAG AA contrast, one button system, the 360 px layout, and real loading, empty and error states.
- **D2, taste pass:** checked with a real-browser design inspector. It removed hairlines, outlined boxes and resting shadows, replaced emoji with our own SVG icons, set a display-size headline, and added a short staggered entrance (off under reduced motion).
- **D3, theme:** Chalkboard and Paper with serif headings in all three scripts. Every colour pair was checked, and darker "ink" shades were added where a colour failed as text.



The one-page A4 summary (Hindi), ready to print or download as a PDF.

8 Models, voices, fonts and licences

Every model, voice, font and tool is listed with its licence in `LICENSES.md` before it was committed. All of them run locally.

Item	Used for	Licence
Meta Llama 3.2 3B Instruct (via Ollama), **Built with Llama**	Lesson planner	Llama 3.2 Community License + Acceptable Use Policy
Ollama	Local model server	MIT
AI4Bharat IndicTrans2 en-indic dist-200M	Translation en→hi, ta	MIT
Kokoro-82M (ONNX), kokoro-onnx	English and Hindi voice	Apache-2.0 / MIT
eSpeak NG (in espeakng-loader)	Phonemes for Kokoro	GPL-3.0-or-later (loader MIT)
AI4Bharat Indic-TTS Tamil (FastPitch + HiFi-GAN), coqui-tts	Tamil voice (default)	MIT / MPL-2.0
AI4Bharat Indic Parler-TTS, parler-tts	Tamil voice (opt-in)	Apache-2.0
Noto Sans, Sans Mono, Sans Devanagari, Sans Tamil, Serif, Serif Devanagari, Serif Tamil	All text in the app and video	SIL Open Font License 1.1
FFmpeg + libx264	MP4 encoding, ffprobe checks	GPL (run as a separate program)
Playwright + Chromium	Headless capture for the MP4	Apache-2.0 / BSD-3-Clause
PyTorch (CPU), Transformers, ONNX Runtime	Model runtimes	BSD-3-Clause / Apache-2.0 / MIT
FastAPI, Pydantic, React, Vite, TypeScript and others	Backend and frontend	MIT / BSD / Apache-2.0 (full list in LICENSES.md)

How to run it

One-time online setup: install Python 3.11+, Node 20+, FFmpeg and Ollama, then the dependencies, then `ollama pull llama3.2:3b` and `scripts/download_models.py` (about 7 GB of models). After that, with Wi-Fi off:

`python scripts/check_offline.py` → `uvicorn app.main:app --app-dir backend --port 8000` → open `http://localhost:8000`.

`scripts/doctor.py --smoke` checks a fresh install. `scripts/render_sample.py` renders a bundled lesson without the planner. `scripts/drill.py` runs the failure drills.

9 Known limitations and next steps

Limitations, stated honestly

- **A 3B planner writes simple lessons.** Facts are not verified. Long lessons can come out short: at 90 s intermediate, real plans had a median of 58% of the narration asked for. A short plan is now kept as a shorter lesson, with an honest notice, instead of falling back.
- **Machine translation.** Live Hindi and Tamil come from IndicTrans2 and can be formal or wrong. The bundled Tamil samples were rewritten by hand in spoken Tamil; the Hindi text still needs a native speaker's check.
- **Quiz answers are not fact-checked.** They are checked for shape and markup only. In testing, one question had a wrong "right" answer.
- **One lesson at a time.** Requests queue, by design, so that only one model is ever in memory.
- **Not yet done:** the benchmark on the 8 GB laptop, the offline matrix with Wi-Fi physically off, and a check on a real phone.

Next steps

- **Longer, deeper plans from a small model.** We found four likely causes in the prompt:
  - a "2 to 3 short sentences" rule that contradicts the word budget;
  - a single 45 s example that the model copies;
  - word counts, which a small model cannot follow;
  - no extra depth asked for at the intermediate level.

  The fix is to ask for a number of sentences per scene, add level-specific guidance and make the retry concrete, then measure it with real runs.
- **More Indian languages** (IndicTrans2 already covers 22), and a native-speaker review loop for the bundled translations.
- **More scene types** (timelines, comparisons, simple charts) drawn by our own components, with the same data-not-code rule.
- **A small classroom mode:** the same laptop serving lessons to phones on the local network, still fully offline.

> **In one line:** OffGridGuru turns a typed concept into a narrated, captioned, animated lesson in English, Hindi or Tamil, on a CPU-only laptop, fully offline, and it always plays, even when a model fails.

Team Rogue · Bhuvaneshwer S and Jeyanth S · OffGridGuru (CodeStorm24) · github.com/Jeyxnth/OffGridGuru